

Traditional Image Processing Techniques

Image Loading and Conversion

cv2.imread(path, flags)

- Loads an image from a file.
- cv2.IMREAD_COLOR (default): Loads a color image in BGR.
- cv2.IMREAD_GRAYSCALE: Loads image in grayscale.
- cv2.IMREAD_UNCHANGED: Includes alpha channel if present.

cv2.cvtColor(image, code)

- Converts color space of the image.
- cv2.COLOR_BGR2GRAY: Color to grayscale
- cv2.COLOR_BGR2HSV: BGR to Hue-Saturation-Value

cv2.resize(image, dsize, interpolation)

- Resizes image to specified width & height. Interpolation options:
- cv2.INTER_NEAREST: Nearest neighbor
- cv2.INTER_LINEAR: Bilinear (default)
- cv2.INTER_CUBIC, cv2.INTER_AREA: Higher quality resizing

cv2.threshold(src, thresh, maxval, type)

- Applies global thresholding to an image.
- Convert grayscale to binary or other maps.
- cv2.THRESH_BINARY, cv2.THRESH_TRUNC, cv2.THRESH_TOZERO, etc.

cv2.adaptiveThreshold(image, maxValue, adaptiveMethod, thresholdType, blockSize, C)

- Computes threshold per region of the image.
- Useful when lighting is uneven.
- adaptiveMethod: cv2.ADAPTIVE_THRESH_MEAN_C, cv2.ADAPTIVE_THRESH_GAUSSIAN_C

cv2.normalize(src, dst, alpha, beta, norm_type)

- Normalizes pixel values between alpha and beta.
- Useful for contrast stretching or preparing data for ML.

Noise Removal / Smoothing

cv2.blur(image, ksize)

- Averaging filter – replaces each pixel with the mean of its neighborhood.
- ksize: kernel size like (3,3)

cv2.GaussianBlur(image, ksize, sigmaX)

- Gaussian filter – smooths image while preserving edges better than average.
- Good for reducing Gaussian noise.

cv2.medianBlur(image, ksize)

- Median filter – replaces pixel with the median of surrounding pixels.
- Excellent for removing salt-and-pepper noise.

cv2.bilateralFilter(src, d, sigmaColor, sigmaSpace)

- Smooths image while preserving edges.
- d: neighborhood diameter.
- Good for denoising photos without losing texture.

Image Enhancement

cv2.equalizeHist(image)

- Improves contrast by spreading out pixel intensity distribution.
- Only works with 8-bit grayscale images.

cv2.createCLAHE(clipLimit, tileGridSize)

- CLAHE = Contrast Limited Adaptive Histogram Equalization.
- Better for local contrast enhancement without over-saturation.

cv2.convertScaleAbs(src, alpha, beta)

- Adjust brightness and contrast: $\text{dst} = \text{saturate}(|\alpha \times \text{src} + \beta|)$

Gamma Correction (manual): $\text{image_corrected} = \text{np.power}(\text{image}/255.0, \text{gamma}) * 255$

- Gamma < 1: brightens, Gamma > 1: darkens.

Morphological Operations

cv2.erode(src, kernel)

- Shrinks white regions (foreground).
- Removes small white noise.

cv2.dilate(src, kernel)

- Expands white regions.
- Fills small black holes.

cv2.morphologyEx(src, op, kernel)

- Combines erosion & dilation.
- cv2.MORPH_OPEN: erosion $\rightarrow$ dilation (remove noise)
- cv2.MORPH_CLOSE: dilation $\rightarrow$ erosion (fill gaps)
- cv2.MORPH_GRADIENT: difference between dilation and erosion

cv2.getStructuringElement(shape, ksize)

- Creates a kernel used in morphology. cv2.MORPH_RECT, MORPH_ELLIPSE, MORPH_CROSS

Edge Detection

cv2.Canny(image, threshold1, threshold2)

- Multistage edge detection using gradient and hysteresis.
- Powerful and widely used.

cv2.Sobel(src, ddepth, dx, dy, ksize)

- Computes gradients in X or Y direction.
- Useful for edge orientation detection.

cv2.Laplacian(src, ddepth)

- Computes 2nd-order derivative (Laplacian).
- Detects edges without direction.

cv2.Scharr(src, ddepth, dx, dy)

- A higher-accuracy version of Sobel for small kernels (3x3).

Geometric Transformations

cv2.getRotationMatrix2D(center, angle, scale)

- Creates affine matrix to rotate around a point.

cv2.warpAffine(image, M, dsize)

- Applies affine transform (translation, rotation, scale).

cv2.getAffineTransform(src_pts, dst_pts)

- Maps 3 points in input to 3 in output.

cv2.getPerspectiveTransform(src_pts, dst_pts)

- Maps 4 points to a new quadrilateral.

cv2.warpPerspective(image, M, dsize)

- Applies perspective warp.

cv2.flip(image, flipCode)

- Flips image: 0: vertical, 1: horizontal, -1: both

cv2.transpose(image)

- Rotates image 90 degrees and flips along diagonal.

Thresholding & Masking

cv2.inRange(image, lower_bound, upper_bound)

- Creates a binary mask where pixel values lie within a range (e.g., for color segmentation).

Contours and Shape Extraction

cv2.findContours(image, mode, method)

- Finds external or internal contours.
- Returns list of points for each contour.

cv2.drawContours(image, contours, index, color)

- Draws contours onto an image.

cv2.boundingRect(contour)

- Gets a rectangle around the contour.

cv2.minAreaRect(contour)

- Rotated bounding rectangle with minimal area.

cv2.connectedComponents(image)

- Labels each connected component in a binary image.

Color Channel Manipulation

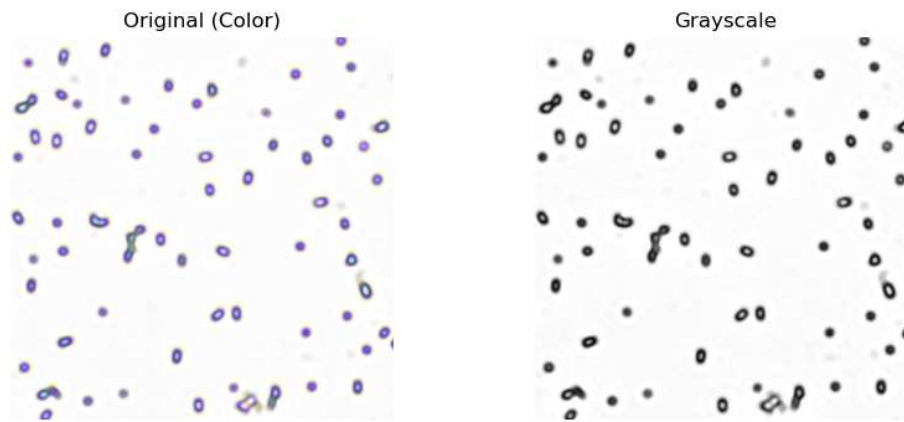
cv2.split(image)

- Splits image into individual channels (e.g., B, G, R).

cv2.merge([channels])

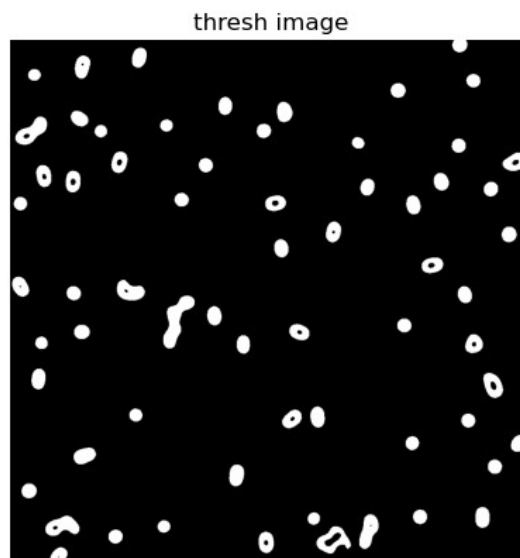
- Combines channels back into a multi-channel image.

Example:



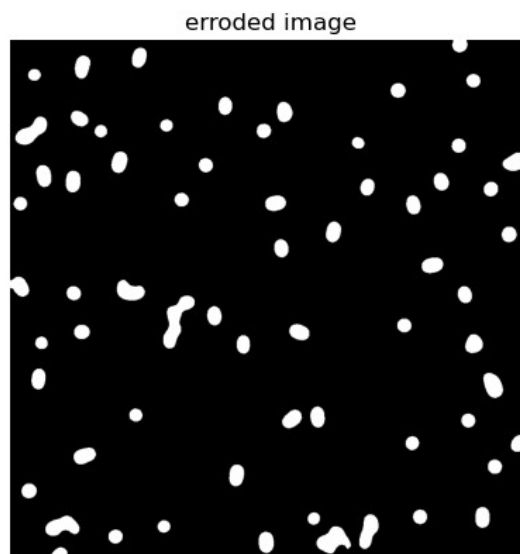
Perform OTSU thresholding, which separates foreground from background:

```
ret, thresh = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY_INV+cv2.THRESH_OTSU)
```



Apply morphological closing: Dilation followed by erosion:

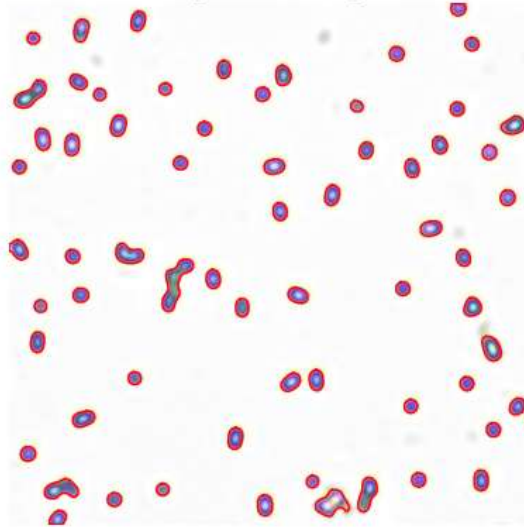
```
kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (15, 15))  
closed = cv2.morphologyEx(thresh, cv2.MORPH_CLOSE, kernel)
```



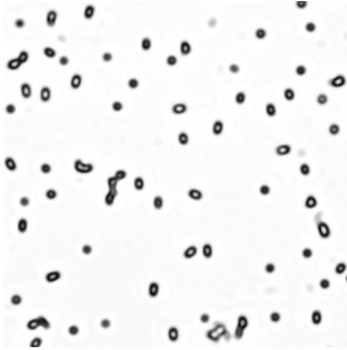
Find contours:

```
contours, _ = cv2.findContours(closed, cv2.RETR_TREE, cv2.CHAIN_APPROX_SIMPLE)
```

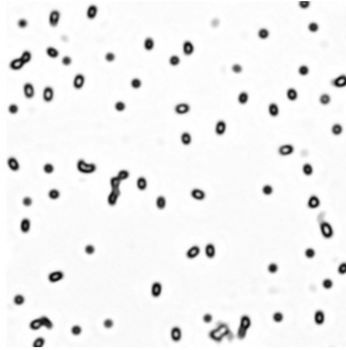
Segmented Image



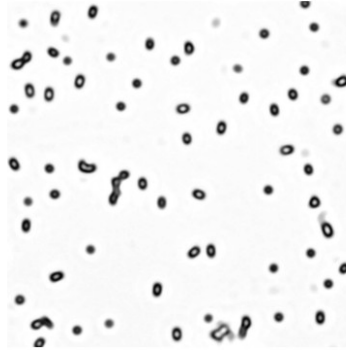
Gray Scale



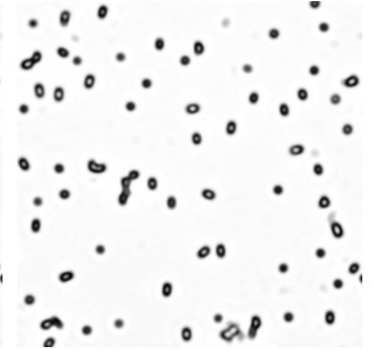
Gaussian Blur



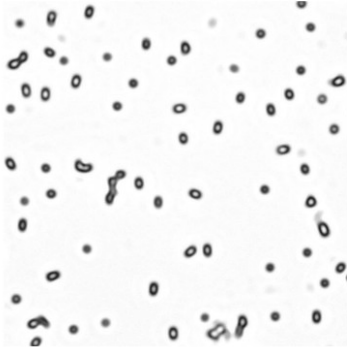
Median Blur



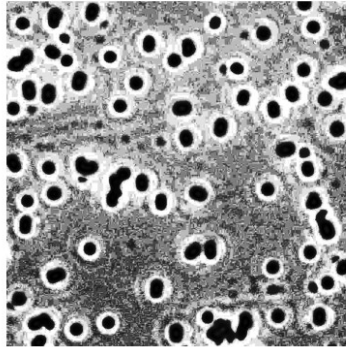
Bilateral Filter



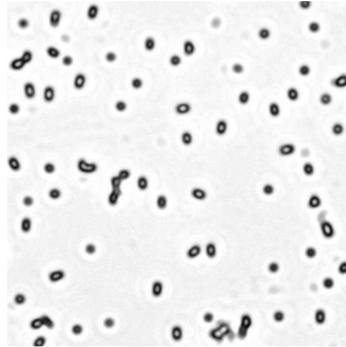
Sharpened



Equalized



CLAHE



1. Global Features

a. Color Features

- Color Histogram: Counts pixel intensities per color channel (RGB, HSV, Lab).
- Color Moments: Mean, variance, skewness per channel.
- Color Coherence Vector (CCV): Combines color histogram with spatial coherence.
- Dominant Color Descriptor (DCD): MPEG-7 feature capturing major colors.

b. Texture Features

- Gray-Level Co-occurrence Matrix (GLCM): Captures spatial relationships of pixel intensities.
 - Extracts: contrast, correlation, energy, homogeneity.
- Local Binary Pattern (LBP):
 - Thresholds neighbors of a pixel; forms binary codes. Effective for texture classification.
- Gabor Filters:
 - Bank of filters at different scales and orientations. Captures spatial frequency and orientation.
- Wavelet Transform:
 - Decomposes image into multi-resolution frequency sub-bands (e.g., Haar, Daubechies).
 - Laws' Texture Energy: Applies convolution kernels to detect texture energy across regions.

2. Local Features

These describe small key point regions in an image, useful for matching and recognition.

a. Corner/Interest Point Detectors

- Harris Corner Detector: Measures local autocorrelation matrix to detect corners.
- Shi-Tomasi Corner Detector: Refinement of Harris; selects best “good features to track.”
- FAST (Features from Accelerated Segment Test): Very fast keypoint detection based on brightness difference.

b. Blob Detectors

- Laplacian of Gaussian (LoG): Detects blobs using scale-space analysis.
- Difference of Gaussian (DoG): Approximation of LoG used in SIFT.
- Determinant of Hessian: Used in SURF.

c. Keypoint Descriptors

- SIFT (Scale-Invariant Feature Transform): Detects key points, then describes them based on gradient orientation histograms. Scale and rotation invariant.
- SURF (Speeded-Up Robust Features): Faster approximation of SIFT using box filters.
- ORB (Oriented FAST and Rotated BRIEF): Binary descriptor; good for real-time and mobile systems.
- BRIEF, FREAK, BRISK: Binary descriptors; fast and compact.

3. Shape Features

Capture object boundaries, contours, and geometries.

a. Edge and Contour-Based

- Canny Edge Detector: Multi-stage edge detection algorithm.
- Hough Transform: Finds lines or circles in binary images.
- Fourier Descriptors: Describes contour shape in frequency domain.
- Chain Code: Encodes direction of boundary traversal.
- Curvature Scale Space (CSS): Based on inflection points on curves.

b. Region-Based

- Zernike Moments: Shape descriptor using orthogonal polynomials.

- Hu Moments: 7 moment invariants that are translation, scale, and rotation invariant.

4. Keypoint Matching and Descriptor Distance Metrics

- Euclidean Distance: For real-valued descriptors (SIFT, SURF).
- Hamming Distance: For binary descriptors (ORB, BRIEF).
- RANSAC: Robust estimation to reject outlier matches.
- FLANN: Fast approximate nearest neighbor search.

5. Feature Selection & Dimensionality Reduction

- Principal Component Analysis (PCA): Projects data to lower dimension.
- Linear Discriminant Analysis (LDA): Maximizes class separability.
- t-SNE / UMAP (for visualization): Maps features to 2D or 3D.

6. Bag of Visual Words (BoVW)

- Treats features like “words” in NLP:
 - Extract local descriptors (e.g., SIFT).
 - Cluster them into “visual words” using K-means.
 - Represent image as histogram over visual words.
- Often used with SVM classifiers.